

Model Format Conversion

Model Format Conversion

1. Model Format Introduction
 - Common Model Formats
2. Course Content
3. Convert Model
 - 3.1 Model Conversion Program
 - 3.1 Start Conversion
4. Source Code Analysis
 - 4.1 Model Conversion Program

[!IMPORTANT]

- The factory image already includes converted format traffic sign and lane detection models. This tutorial is provided for users who need to master model conversion methods
- If you want to experience the complete autonomous driving functionality directly, you can skip this section!!!

1. Model Format Introduction

- The ultimate goal of training models is to deploy them in practical applications. The export modes in YOLOv11 provide multiple options to export trained models into different formats, making them deployable on various platforms and devices.



Common Model Formats

- PyTorch (.pt) : Default native format, containing complete model structure and weights. Applicable scenarios: Model training, fine-tuning, foundation for subsequent format export, not suitable for direct deployment.
- TorchScript (.torchscript) : Serialized format of PyTorch models, stripped of Python dependencies. Applicable scenarios: Production environment deployment in PyTorch ecosystem (such as server-side inference), supports C++ calling.

- ONNX (.onnx) : Open Neural Network Exchange format, the core standard for cross-framework compatibility.
Applicable scenarios: CPU/GPU universal deployment, can achieve 3x CPU acceleration with ONNX Runtime.
- TensorRT (.engine) : NVIDIA exclusive optimized format, GPU-based inference engine.
Applicable scenarios: Real-time inference on NVIDIA GPUs (such as Jetson development boards), can achieve 5x GPU acceleration.
- OpenVINO (openvino_model) : Intel exclusive format, optimized for Intel CPU/GPU/VPU.
Applicable scenarios: Deployment on Intel hardware (such as PCs, edge computing devices).
- TF Lite (.tflite) : TensorFlow lightweight format, targeting mobile/edge devices.
Applicable scenarios: Android devices, embedded hardware (such as Raspberry Pi), supports INT8 quantization compression.
- RKNN (rknn_model) : Rockchip exclusive format, adapted to its AI chips.
Applicable scenarios: Deployment on Rockchip RK series chips (such as development boards, smart hardware).
- Comparison of Different Format Models

Comparison	Universal Format (ONNX)	Accelerated Format (TensorRT)	Edge Format (TF Lite)	Native Format (.pt)
Inference Speed	Medium (CPU/GPU Universal)	Extremely Fast (GPU Exclusive)	Low - Medium (Lightweight)	Low (Python Dependent)
Model Size	Medium	Medium (supports quantization compression)	Extremely Small (INT8 Quantization)	Large
Hardware Dependency	None	NVIDIA GPU	Mobile / Embedded Devices	PyTorch Environment
Applicable Scenarios	Universal Deployment, Cross-Framework	Real-time High-Concurrency Inference	Low-Power Edge Devices	Training / Fine-Tuning
Precision Support	FP32/FP16	FP32/FP16/INT8	FP32/FP16/INT8	Full Precision

2. Course Content

1. Convert pt model (native format exported by YOLOv11) to TensorRT format model and compare the inference speed differences between the two

What are the benefits of using TensorRT to export models?

Using TensorRT to export models on the Jetson platform can significantly improve model inference performance. YOLO11 models exported to TensorRT can achieve up to 5x GPU acceleration

- **Speed:** Faster inference through advanced optimizations.
- **Compatibility:** Smooth integration with NVIDIA hardware.

3. Convert Model

[!NOTE]

- Only Orin boards are recommended for on-board model training. The training source code is in the yolo_train folder under Model Training in the source code summary. If training is needed, you can place the yolo_train folder programs in the root directory of the board
- **RDk Mainboard Conversion Model Reference [YOLOV11 Development (RDKX5)] - [Model Quantization]**

3.1 Model Conversion Program

Model format conversion program path:

```
/home/jetson/yolo_train/conver_model.py
```

Optional Launch Parameters Description:

- **format:** Model format to export, default is engine (TensorRT format)
- **MODEL_PATH** Original pt format model path
- **data** Calibration dataset, only needed when performing int8 quantization on the model
- **device** Running device number, default is 0
- **int8** Whether to use int8 quantization to further compress model size, default is False

3.1 Start Conversion

```
cd ~/yolo_train
```

- Here we use the traffic sign recognition model trained in the previous section as an example for conversion (here replace the --MODEL_PATH path with your actual named .pt model path):

```
python3 conver_model.py --MODEL_PATH /home/jetson/yolo_train/yolo11n.pt
```

- After model conversion and export is completed, the terminal prompts `Export complete` and the exported model path

```

jetson@yahboom:~/yolo_train$ python3 conver_model.py --MODEL_PATH /home/jetson/yolo_train/yolo11n.pt
t
Ultralytics 8.3.59 Python-3.10.12 torch-2.5.0a0+872d972e41.nv24.08 CUDA:0 (Orin, 3602MiB)
YOLO11n summary (fused): 238 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs

PyTorch: starting from '/home/jetson/yolo_train/yolo11n.pt' with input shape (1, 3, 640, 640) BCHW
and output shape(s) (1, 84, 8400) (5.4 MB)

ONNX: starting export with onnx 1.17.0 opset 19...
ONNX: slimming with onnxslim 0.1.47...
ONNX: export success 2.7s, saved as '/home/jetson/yolo_train/yolo11n.onnx' (10.2 MB)

TensorRT: starting export with TensorRT 10.7.0...
[12/01/2025-21:21:47] [TRT] [I] [MemUsageChange] Init CUDA: CPU +2, GPU +0, now: CPU 640, GPU 2861 (MiB)
[12/01/2025-21:21:52] [TRT] [I] [MemUsageChange] Init builder kernel library: CPU +980, GPU +537, now: CPU 1664, GPU 3414 (MiB)
[12/01/2025-21:21:52] [TRT] [I] -----
[12/01/2025-21:21:52] [TRT] [I] Input filename: /home/jetson/yolo_train/yolo11n.onnx
[12/01/2025-21:21:52] [TRT] [I] ONNX IR version: 0.0.9
[12/01/2025-21:21:52] [TRT] [I] Opset version: 19
[12/01/2025-21:21:52] [TRT] [I] Producer name: pytorch
[12/01/2025-21:21:52] [TRT] [I] Producer version: 2.5.0
[12/01/2025-21:21:52] [TRT] [I] Domain:
[12/01/2025-21:21:52] [TRT] [I] Model version: 0
[12/01/2025-21:21:52] [TRT] [I] Doc string:
[12/01/2025-21:21:52] [TRT] [I] -----
TensorRT: input "images" with shape(1, 3, 640, 640) DataType.FLOAT
TensorRT: output "output0" with shape(1, 84, 8400) DataType.FLOAT
TensorRT: building FP32 engine as /home/jetson/yolo_train/yolo11n.engine
[12/01/2025-21:21:53] [TRT] [I] Local timing cache in use. Profiling results in this builder pass will not be stored.

```

- The best.engine in the output directory here is the exported TensorRT format model file

```

jetson@yahboom:~/yolo_train$ ls
conver_model.py  train_lane  train.py  train_sign  yolo11n.engine  yolo11n.onnx  yolo11n.pt
jetson@yahboom:~/yolo_train$

```

[!WARNING]

- Attention!!! TensorRT format models converted and exported on different GPU platforms are not universal due to different TensorRT versions.
- For example, TensorRT format models converted and exported on Jetson Orin NX cannot be used on Jetson Nano because the TensorRT versions on the two platforms are different.

4. Source Code Analysis

4.1 Model Conversion Program

The program source code is located at `/home/jetson/yolo_train/conver_model.py`

The program process is divided into:

- Parse parameters: `args = get_args()` reads parameters passed from command line, such as model path, export format, etc.
- Load model: `model = YOLO(args.MODEL_PATH)` loads YOLO pre-trained model from specified path.
- Export model: Call different export configurations based on whether INT8 quantization is enabled

```

import argparse
from ultralytics import YOLO

def get_args():

```

```

'''
'''

parser = argparse.ArgumentParser()
parser.add_argument('--project', default='./project', help='Project file
output directory', type=str)
parser.add_argument('--
MODEL_PATH', default='/home/jetson/MODELS/yolov11/yolo11n.pt' ,help='原始模型权重文
件路径Original model weight file path', type=str)
parser.add_argument('--epochs', default=100, help='训练轮次Training rounds',
type=int)
parser.add_argument('--imgsz', default=640, help='图像大小Image size',
type=int)
parser.add_argument('--train_config', default='', help='配置文件路径
Configuration file path', type=str)
return parser.parse_args()

if __name__ == "__main__":
    args = get_args()
    model = YOLO(args.MODEL_PATH) # load a pretrained model

    results = model.train(data=args.train_config, epochs=args.epochs,
imgsz=args.imgsz,project=args.project)#start training

```